An Authoritative STM32F051 Tic-Tac-Toe Server with a Thin Python Client

Line-framed ASCII protocol, server-side validation and an automated board test suite

Sibongokuhle Sibiya (SBYSIB014)
EEE3095S CS Project 2026, Department of Electrical Engineering
University of Cape Town

***Abstract*—I built a two-player Tic-Tac-Toe server on an STM32F051 board that acts as the only referee, and a Python/Tkinter client that forwards clicks and draws what the server confirms. The two sides talk over the board's ST-Link virtual COM port (USART1, 115200 baud, 8N1) using newline-terminated ASCII frames of at most 63 characters. The server checks every request in a fixed order before it writes any state, so a rejected request leaves the game unchanged. It answers malformed, oversized, non-printable and incomplete frames with specific `NACK` codes. I added three optional extensions: a CRC-8 frame checksum, an AI opponent that moves through the same function as a human, and move history with undo. A pytest harness drives the real board without human input: 23 core tests and 24 extension tests. On the first run on the board, 21 of the 23 core tests passed (the extension tests did not exist yet). The two failures exposed UART overruns in the HAL receive path, which I replaced with a register-level interrupt handler. After that fix, all 47 tests passed on the board in each of five runs, four with plain frames and one with a checksum on every request.**

I. Introduction

A serial game has two programs that can disagree about the state of the game. If the PC is trusted, a faulty or hostile client can place two marks in one turn, overwrite an occupied cell or keep playing after the game has ended, and the board and the screen then show different games. The project brief [1] therefore makes the STM32 the referee. My design guarantees one property: a request that the server rejects never changes the game state.

I chose Tic-Tac-Toe, the baseline game in the brief, so that my effort went into the protocol, the robustness of the server and the automated tests rather than into game rules. The client uses Tkinter [2] because it ships with the standard Python installer and needs no extra installation, and pySerial [3] for the serial port.

Beyond the compulsory system I built three of the optional extensions that the brief lists or recommends: a CRC-8 checksum on every frame, an AI opponent, and move history with undo. Each one sits behind new commands, so the compulsory protocol behaves exactly as before and the 23 core tests still apply.

Three design choices shape the system. First, the firmware splits reception in two: a short interrupt handler stores each byte in a ring buffer, and the main loop assembles lines, parses them and runs the state machine. Second, the protocol is line-framed ASCII with no spaces, so a person can test the server from a serial terminal. Third, one pytest harness drives the real board without human interaction, and every robustness rule has a named test.

TABLE I
Project plan: milestones and outputs.

	Milestone and output	Date
1	Protocol and server FSM specified	16/10/2026
2	Firmware: framing, state machine, validation	16/10/2026
3	Tkinter GUI client	16/10/2026
4	Test harness, 23 core tests; first board run	24/10/2026
5	Receive-path fix after the failed tests	24/10/2026
6	Extensions: checksum, AI, undo, 24 more tests	24/10/2026
7	Final board runs, report, video, repository	26/10/2026

Task allocation and plan. The brief allows teams of two; I worked alone, so I did all of the work: protocol, firmware, GUI, test harness, extensions and this report. Table I lists the milestones in the order I worked.

Structure. Section II defines the requirements, the message protocol and the extensions. Section III presents the server FSM, the GUI flowchart and the circuit. Section IV explains five code excerpts. Section V argues that the server stays authoritative. Section VI covers framing and recovery from bad input. Section VII reports the tests on the board, and Section VIII concludes.

II. Requirements and Message Protocol

A. Newline-terminated frames of at most 63 characters

The link is USART1 on PA9 (TX) and PA10 (RX), routed through the on-board ST-Link virtual COM port [4]: 115200 baud, 8 data bits, no parity, 1 stop bit, no flow control. Table II lists the framing rules. Every rule has one reply, so a client can tell why a frame was refused.

B. An optional CRC-8 suffix catches errors that still look like commands

Any frame may end in `*HH`, two hex digits of a CRC-8 (polynomial 0x07, initial value 0, no reflection) over everything before the asterisk; for example `MOVE:4` becomes `MOVE:4*75`, and the standard check value is `crc8("123456789") = 0xF4`. The server verifies a suffix before it parses the command and sends `NACK:CHECKSUM` on a mismatch. This catches a bit error that turns one valid

TABLE II
FRAMING RULES AND THE RESPONSE TO EACH VIOLATION.

Rule	Server behaviour
Terminator	`\n` or `\r` ends a frame; an empty frame is ignored, so `\r\n` gives one reply
Maximum length	63 characters; the rest of the line is discarded up to the next terminator; one `NACK:TOOLONG`
Incomplete frame	no terminator for 500 ms after the last byte: frame discarded, `NACK:TIMEOUT`
Invalid character	any byte outside 0x20–0x7E: frame discarded, `NACK:BADCMD`
UART fault	overrun, framing or noise error, or a full 128-byte receive ring: frame discarded, `NACK:BADCMD`
Checksum	optional suffix `*HH` (CRC-8); a wrong value gives `NACK:CHECKSUM` and nothing runs

TABLE III
COMMAND SET. † MARKS A DEPARTURE FROM THE BRIEF.

Request	Accepted in	Success response
`CONNECT`	any state†	`OK` (opens the session)
`NEW`	IDLE, GAME_OVER	`OK:BOARD:<9>`
`NEW:AI`†	IDLE, GAME_OVER	`OK:BOARD:<9>`; the AI plays O
`MOVE:<i>`	IN_PROGRESS	`OK:BOARD:<9>`, `WIN:X`, `WIN:O` or `DRAW`; in AI mode the board after the AI's reply
`MOVE:<i>:<m>`†	IN_PROGRESS	as above; `<m>` is X or O
`STATE`	IN_PROGRESS, GAME_OVER	`BOARD:<9>`
`HISTORY`†	IN_PROGRESS, GAME_OVER	`HIST:<i>,<i>,...`
`UNDO`†	IN_PROGRESS	`OK:BOARD:<9>`
`RESIGN`	IN_PROGRESS	`OK:GAMEOVER`
`DISCONNECT`	any state†	`OK` (returns to IDLE)
Button press†	session open	unsolicited `OK:BOARD:.........`

command into another, such as `MOVE:4` into `MOVE:5`, which the framing rules cannot see. A reply carries a suffix exactly when the request did, so a client that sends plain frames is unaffected. Text that only looks like a suffix, such as `NEW*ZZ`, is not a checksum and is rejected as `NACK:BADCMD`.

C. Nine characters encode the board

The board is nine characters, cells 0 to 8 row by row, using `.` for empty, `X` and `O`. For example, `XO.XO....` is X in cells 0 and 3, O in cells 1 and 4, and everything else empty. X always moves first.

D. Nine requests and one unsolicited message

Table III lists the command set. The client sends one request at a time and waits for the reply. The only message the server sends on its own is `OK:BOARD:.........`, which follows a new game started with the board's button; the client treats it like the reply to `NEW`, so the display stays in step with the board.

E. Every failure has its own NACK and a fixed check order

Table IV lists every error. When a request could fail several checks, the server uses one order: framing first, then whether

TABLE IV
EVERY ERROR RESPONSE AND WHEN THE SERVER SENDS IT.

Response	Sent when
`NACK:BADCMD`	unknown or wrong-case command, bad `MOVE` syntax, non-printable byte, UART fault, or `NEW` during a game
`NACK:NOGAME`	`MOVE`, `STATE` or `RESIGN` with no live game (IDLE, or `MOVE`/`RESIGN` after game over)
`NACK:RANGE`	index is negative or above 8
`NACK:OCCUPIED`	the cell already holds a mark
`NACK:NOTURN`†	the optional mark is not the side to move
`NACK:NOCONN`†	a known command arrives before `CONNECT`
`NACK:TOOLONG`†	frame longer than 63 characters
`NACK:TIMEOUT`†	frame not terminated within 500 ms
`NACK:CHECKSUM`†	the `*HH` suffix does not match the payload
`NACK:NOUNDO`†	`UNDO` with no move to take back

the command is known, then the session, then the game state, and for `MOVE` the index syntax, the index range, the turn and the cell. A checksum, if present, is checked right after framing. Each request therefore has exactly one correct reply.

F. Departures from the reference protocol

Optional mark on `MOVE`. With one client playing both sides, the server cannot see an out-of-turn move in `MOVE:<i>`. The suffix `:<m>` names the player; the server answers `NACK:NOTURN` if `<m>` is not the side to move, which makes the rule testable. The GUI does not send the suffix. **`CONNECT` and `DISCONNECT` in any state.** A reconnecting client works after a crash, and closing the GUI mid-game always returns the server to IDLE. **Extra errors.** `NOCONN`, `TOOLONG` and `TIMEOUT` give a specific reply to a specific fault instead of one generic error. **Unsolicited board.** The button needs a way to tell the client that a new game began. **Extensions.** The checksum suffix, `NEW:AI`, `HISTORY`, `UNDO` and their replies are optional additions; none of them changes the reply to a compulsory request.

Figure 1 shows one session. After the second `MOVE:0` is refused with `NACK:OCCUPIED`, the game continues from the same board. After the win, `STATE` returns the final board, because the server sends only `WIN:X` and not the board. In step 8 the server discards the whole 103-character line, including the `NEW` at its end.

III. SPECIFICATION AND DESIGN

A. The server decides; the client only draws

Table V assigns each decision to one side. The server holds the only copy of the game state (the board, the side to move and the game status) and the client never computes any of it. The firmware has two layers. Hardware code covers the USART interrupt, polled transmit, the LED and the button. Rule code covers `check_win`, `check_draw`, `handle_move` and the command dispatcher, which depend only on the game variables and on `send_str`. I kept the rules apart from the interrupt and LED code so that they never depend on timing.

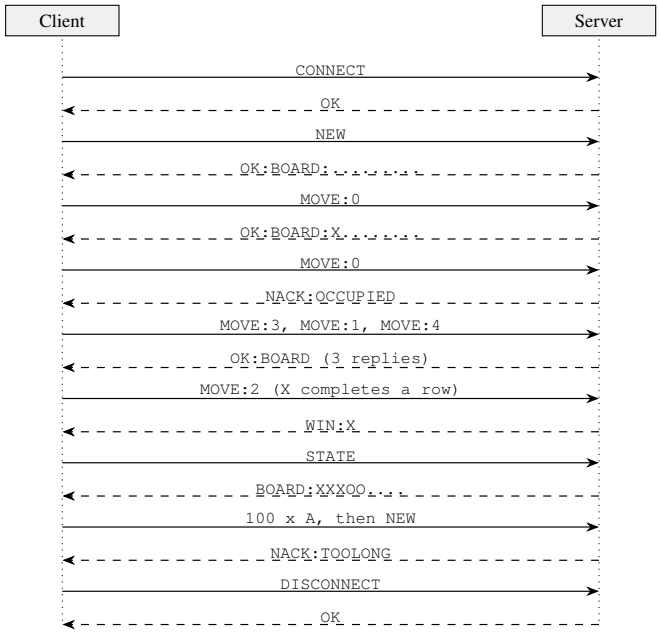


Fig. 1. One session. A rejected move (step 4) is answered with a NACK and the game continues; a win (step 6) ends the game; an oversized line (step 8) is refused without running the `NEW` at its end. Replies are dashed.

TABLE V
WHO OWNS EACH DECISION.

Decision	Owner	Why
Board, turn, game status	server	one source of truth
Is a move legal?	server	a client cannot be trusted
Win and draw detection	server	result must match the LED
Session (`CONNECT`)	server	rejects commands before it
What is drawn	client	only from `OK:BOARD`, `BOARD`
AI move choice	server	same write path as a human
Move history, undo	server	client keeps no move list
LED and button	server	board-side indicators

B. Three states and a validation chain that writes last

Figure 2 shows the server FSM. In IDLE the server accepts `NEW` (or `NEW:AI`) once a session is open. In IN_PROGRESS it accepts `MOVE`, `STATE`, `HISTORY`, `UNDO` and `RESIGN`; a legal move stays in IN_PROGRESS unless it wins or fills the board. In GAME_OVER it accepts `NEW`, `STATE` and `HISTORY`, and a `MOVE` or `UNDO` gets `NACK:NOGAME`. `DISCONNECT` returns any state to IDLE. The lower part of the figure expands the validation branch. The five checks come before the first write to the board, so every `NACK` exit leaves the state unchanged. The figure omits the `NOCONN`, `TOOLONG`, `TIMEOUT` and `CHECKSUM` replies, which belong to the framer and the session check and never reach the state machine.

C. The GUI never blocks and never guesses

Figure 3 shows the client flow. A reader thread blocks on the port and puts each line into a queue; the Tk main loop empties the queue every 50 ms, so the window stays responsive. After the port opens, the client resends `CONNECT` every 600 ms, up to 15 times, so it works whether or not opening the port reset the board. It keeps one request in flight and ignores board clicks until the reply arrives or 2 s pass. It redraws the board only from `OK:BOARD` or `BOARD`. After `WIN` or `DRAW` it sends `STATE` and redraws from the answer. Every request carries a checksum; the client ignores a reply with a wrong one and asks for `STATE` instead of acting on it. A tick box chooses `NEW:AI`, and the Undo and History buttons send one request each and show the server's answer, because the client keeps no move list. If the cable is unplugged, the reader thread ends and the next request times out with "No reply from board"; a failed write sets the status line to the error.

D. One USB cable carries power, programming and the serial link

Figure 4 shows the circuit. USART1 TX (PA9) and RX (PA10) connect to the ST-Link's virtual COM port on the board, so no extra wiring or USB-UART converter is used, and the two share ground through the board. The dual-colour LED D9 sits on PB10 (red) and PB11 (green) behind 150 Ω resistors and lights when the pin is high. Red means X to move, green means O to move, both blinking means game over, and both off means IDLE. Button SW0 on PA0 reads low when pressed (the firmware enables the internal pull-up) and starts a new game if a session is open. On my board the red and green dies are physically swapped, so one constant (`BOARD_RG_SWAPPED`) swaps the two pins in software.

IV. IMPLEMENTATION

The listings in this report are trimmed: I removed comments, blank lines and unrelated declarations.

A. A register-level interrupt keeps up with 115200 baud

Listing 1 is the receive path. At 115200 baud a byte arrives every 87 μs. My first version re-armed `HAL_UART_Receive_IT` after every byte [5]. On the board it answered 6-character commands correctly, but frames of 20 and 103 characters were answered with `NACK:BADCMD` instead of `NACK:RANGE` and `NACK:TOOLONG`, which means the server had flagged a UART fault mid-frame. I did not capture the overrun flag directly, so overrun is my diagnosis from this evidence. The handler now reads the status register and the data register itself [6], which is a few dozen instructions long (I did not measure its run time). It clears the error flags, sets `rx_fault` so that the framer rejects the damaged frame, and pushes the byte into a 128-byte ring buffer. The interrupt only writes `rx_head` and the main loop only writes `rx_tail`, so no lock is needed. The first version also never enabled the USART1 interrupt in the NVIC, so no command arrived at all; `HAL_UART_MspInit` now calls `HAL_NVIC_EnableIRQ`.

B. The framer discards a bad line up to its terminator

Listing 2 turns the byte stream into frames. The obvious approach resets the buffer when a line is too long and carries

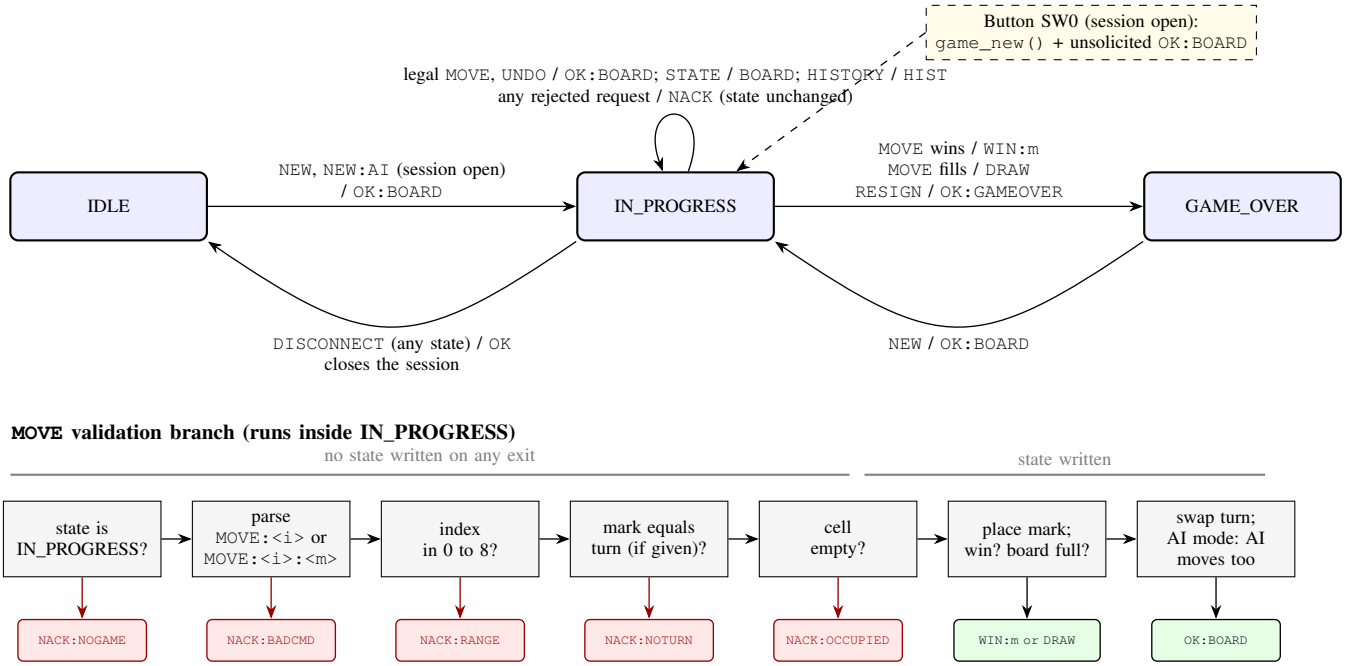


Fig. 2. Server FSM with the move-validation branch. Red exits are NACKs that leave the state unchanged; green exits are successes. The symbol `m` stands for the mark of the side to move.

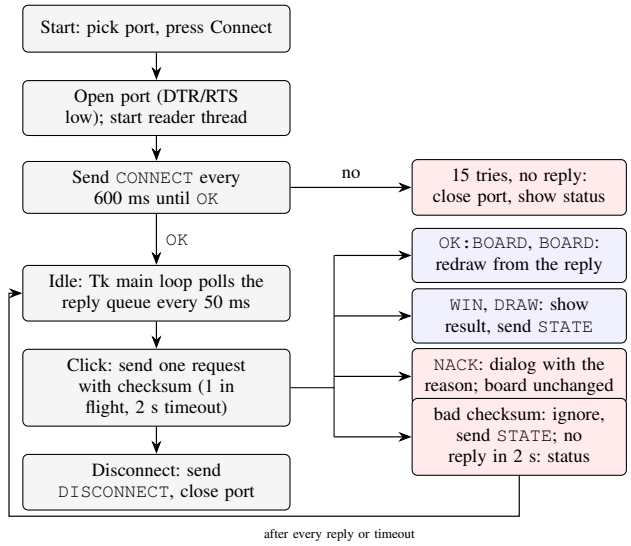


Fig. 3. GUI application flowchart. Every board change comes from a reply line, never from the click itself.

on. Then the end of a 100-character line would be parsed as a new command, so a line ending in `NEW` would start a game. Instead the framer sets `frame_err` and drops every byte until the next terminator, then sends exactly one NACK. A non-printable byte sets the same flag. A separate check in the main loop compares `HAL_GetTick()` with the time of the last received byte and discards a half-received frame after 500 ms, so a lost terminator cannot glue two commands together.

```
void USART1_IRQHandler(void) {
    uint32_t isr = USART1->ISR;
    if (isr & (USART_ISR_ORE | USART_ISR_FE |
               USART_ISR_NE)) {
        USART1->ICR = USART_ICR_ORECF | USART_ICR_FECF
                    | USART_ICR_NCF;
        rx_fault = 1;                /* frame is damaged */
    }
    if (isr & USART_ISR_RXNE)
        uart_rx_push((uint8_t)USART1->RDR);
}
void uart_rx_push(uint8_t b) {
    uint16_t next = (rx_head + 1) % RX_RING_SIZE;
    if (next == rx_tail) rx_fault = 1;  /* ring full */
    else { rx_ring[rx_head] = b; rx_head = next; }
}
```

Listing 1. Interrupt handler and ring buffer (`stm32f0xx_it.c`, `main.c`).

```
while (rx_tail != rx_head) {
    char c = (char)rx_ring[rx_tail];
    rx_tail = (rx_tail + 1) % RX_RING_SIZE;
    last_rx_tick = HAL_GetTick();
    if (c == '\n' || c == '\r') {
        if (frame_err)
            send_str(frame_err == FE_TOOLONG
                ? "NACK:TOOLONG\n" : "NACK:BADCMD\n");
        else if (line_len > 0) {
            line_buf[line_len] = '\0';
            process_command(line_buf);
        }
        line_len = 0; frame_err = FE_NONE;
    } else if (frame_err) { /* drop until terminator */ }
    else if ((unsigned char)c < 0x20 ||
             (unsigned char)c > 0x7E)
        frame_err = FE_GARBAGE;
    else if (line_len < LINE_MAX_LEN)
        line_buf[line_len++] = c;
    else frame_err = FE_TOOLONG;
}
```

Listing 2. Frame assembler (`main.c`).

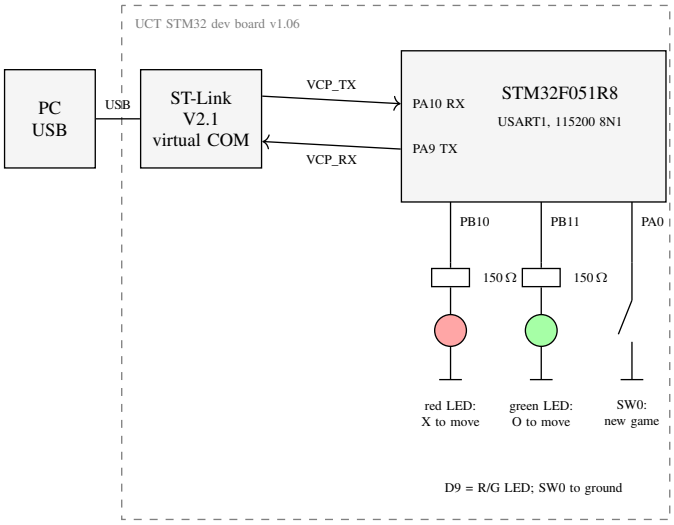


Fig. 4. Circuit: UART link to the PC, the LED and the button.

```
def _read_thread(self, ser):        # worker thread
    while ser.is_open:
        raw = ser.readline()
        line = raw.decode(errors="replace").strip()
        if line:
            self.rx_queue.put(line)

def _poll_queue(self):              # Tk thread
    while not self.rx_queue.empty():
        self._handle_line(self.rx_queue.get())
    self.root.after(50, self._poll_queue)

def _handshake(self):               # until OK
    if not self.ser or self.connected: return
    if self.handshake_tries >= 15:
        self._close_port(); return
    self.handshake_tries += 1
    self._send("CONNECT")
    self.root.after(600, self._handshake)
```

Listing 3. Reader thread, queue polling and handshake (`gui_client.py`).

C. The GUI waits on the port in a thread and draws from a queue

Listing 3 shows how the client stays responsive (I split one line for width). A blocking `readline` in a Tk callback would freeze the window for up to the port timeout on every call. The worker thread is the only code that blocks on the port; it hands lines to the Tk thread through a thread-safe queue, and the Tk thread polls that queue every 50 ms and never blocks. The handshake uses the same timer mechanism, so the GUI keeps sending `CONNECT` while the board is still booting. Tkinter widgets are touched only from the Tk thread [2].

D. The AI searches a copy of the board and moves through the human's write path

Listing 4 is the opponent's search: negamax with alpha-beta pruning, scoring a win as $10 - \mathit{depth}$ so that the AI prefers the quickest win and the slowest loss. Two choices matter. First, the search places and removes marks on a local copy of the board, so the game board changes only in `apply_move`, the single function that writes a mark; the human's `MOVE` and the AI's reply both call it after their own checks, and the AI re-checks that its chosen cell is empty first. A search that wrote to the real board could leave a half-explored position behind. Second, cells are tried centre, corners, then edges, which cuts the search: on a PC build of the same source the worst single decision, the first reply, takes 1 260 calls to `negamax` (mean 35), against up to $8! = 40\,320$ move sequences without pruning. I have not timed the search on the board, but in all five full test runs every AI reply arrived inside the harness's 2 s read timeout. The search runs in the main loop while the receive interrupt keeps buffering bytes.

```
int negamax(char *b, char me, char opp,
            int depth, int alpha, int beta) {
    int best = -100;
    for (int k = 0; k < 9; k++) {
        int i = ai_order[k], sc;   /* 4,0,2,6,8,1,3,5,7 */
        if (b[i] != '.') continue;
        b[i] = me;
        if (line_win(b, me))     sc = 10 - depth;
        else if (board_full(b)) sc = 0;
        else sc = -negamax(b, opp, me, depth + 1,
                           -beta, -alpha);
        b[i] = '.';
        if (sc > best)      best = sc;
        if (best > alpha)   alpha = best;
        if (alpha >= beta) break;
    }
    return best;
}
```

Listing 4. AI search (`main.c`, trimmed).

V. Game Logic and Server Authority

A. Every check returns before the first write

Listing 5 is `handle_move`. It runs the checks in the order of Section II: game state, syntax, range, turn, cell. Each failing check sends one NACK and returns. The parser uses only local variables, and saturates the number at 1000 so that a long digit string cannot overflow; an earlier version used `atoi`, which turned `MOVE:abc` into a move on cell 0.

B. A rejected request cannot change the game state

The game state is six variables: `board`, `turn`, `game_state`, `connected`, `ai_mode` and the move history. The argument has five steps, and each step can be checked against `main.c`.

1) Only seven pieces of code write those variables: `game_new()`, `apply_move()`, `handle_undo()`, and the `CONNECT`, `DISCONNECT`, `NEW` and `RESIGN` branches of `process_command`. Every other path calls only `send_str`.
2) *Framing and checksum errors.* `process_rx` calls `process_command` only for a complete frame with `frame_err` equal to zero and a checksum that is absent or correct. The oversize, garbage, fault, timeout and `CHECKSUM` branches send one NACK and reset `line_len` and `frame_err`, which belong to the framer, not to the game.

```
void handle_move(const char *p) {
    if (game_state != STATE_IN_PROGRESS)
        { send_str("NACK:NOGAME\n"); return; }
    int neg = 0, nd = 0, idx = 0; char mark = 0;
    if (*p == '-') { neg = 1; p++; }
    while (*p >= '0' && *p <= '9') {
        if (idx < 1000) idx = idx*10 + (*p - '0');
        p++; nd++;
    }
    if (nd == 0) { send_str("NACK:BADCMD\n"); return; }
    if (*p == ':') {
        mark = p[1];
        if ((mark != 'X' && mark != 'O') || p[2] != '\0')
            { send_str("NACK:BADCMD\n"); return; }
    } else if (*p != '\0')
        { send_str("NACK:BADCMD\n"); return; }
    if (neg || idx > 8)
        { send_str("NACK:RANGE\n"); return; }
    if (mark && mark != turn)
        { send_str("NACK:NOTURN\n"); return; }
    if (board[idx] != '.')
        { send_str("NACK:OCCUPIED\n"); return; }
    int r = apply_move(idx);  /* first write to the board */
    if (r == MV_CONTINUE && ai_mode) r = ai_reply();
    send_move_result(r);      /* WIN, DRAW or OK:BOARD */
}
```

Listing 5. Move validation (`main.c`, trimmed).

3) *Unknown command or no session.* `process_command` returns with `NACK:BADCMD` or `NACK:NOCONN` before it reaches any handler.
4) *Handlers.* `NEW` returns with `NACK:BADCMD` before it sets `ai_mode` or calls `game_new()` when a game is running. `STATE` and `HISTORY` only read. `RESIGN` returns with `NACK:NOGAME` before it writes `game_state`. `handle_undo` checks the game state and the history length before it removes anything.
5) *`MOVE`.* In Listing 5 the call `apply_move(idx)` is the first write, and every NACK path returns before it.

Hence every path that ends in a NACK has written no game variable. This is an argument from reading the code, not a formal proof, and the tests below sample it. The button is not a request over the link: it is a local override that resets the game when a session is open, and is documented as such. `test_occupied_move_rejected_and_board_unchanged`, `test_malformed_move_does_not_touch_state`, `test_new_during_game_rejected_state_unchanged`, `test_out_of_turn_rejected`, `test_wrong_checksum_rejected_and_nothing_executed` and `test_undo_all_the_way_back_then_nothing_to_undo` each send a bad request and then read `STATE` to compare the board. The oversize and garbage tests check that a bad frame does not run the command at its end.

C. The server alternates the turn and can check a named player

The variable `turn` starts as X and flips only after a legal move that neither wins nor draws. A plain `MOVE:<i>` is played by the side to move. A `MOVE:<i>:<m>` with the wrong mark gets `NACK:NOTURN` before the cell is examined, and `test_out_of_turn_rejected` checks that the board is unchanged. In AI mode the human is always X, so a mark `O` on a `MOVE` is also refused with `NACK:NOTURN`.

D. The AI cannot make an illegal move

`ai_choose` considers only empty cells on a copy of the board, and `ai_reply` checks `board[idx] == '.'` again before it calls `apply_move`, the same function a human `MOVE` ends in. A legal AI move is therefore true by construction, not by testing alone. Two checks back it up. On a PC build of the same `main.c` I enumerated every possible human strategy against the AI: 521 games, the human won none, the AI won 368 and 153 were draws. `test_ai_never_makes_an_illegal_move_and_never_loses` plays 20 seeded random games on the board and checks that each reply differs from the previous board by exactly the human's X and one new O.

E. Undo restores the board and the turn from the history

The server stores the cell of every move. `UNDO` clears the last cell (the last two in AI mode, so it is the human's turn again) and sets the turn from the parity of the history length, so the turn can never disagree with the board. It is refused after game over, because the result has already been sent, and with an empty history it returns `NACK:NOUNDO`. `test_undo_restores_board_and_turn` and `test_undo_in_ai_mode_removes_both_moves` check both modes.

F. Win is checked before draw, for the mover only

After placing a mark the server tests the eight lines (three rows, three columns, two diagonals) for the mover's mark only, because only that mark can have just completed a line. The win test runs before the draw test, so a ninth move that completes a line is a win and not a draw. Either result sets `GAME_OVER`, after which `MOVE` returns `NACK:NOGAME`. `test_scripted_win_x`, `test_scripted_win_o` and `test_scripted_draw` cover these results. No test yet covers a win on the ninth move, and Section VIII lists this as a limitation.

G. The client holds no rules

The GUI sends `MOVE:<i>` for any click, including a click on an occupied cell, and shows the server's reason. If the client also checked legality, two rule sets would have to stay identical, and they would disagree whenever the button started a game that the client did not see. My own first GUI inferred the final mark after a `WIN` from the move count. I removed that guess, and the GUI now asks for `STATE`.

VI. Protocol Robustness and Integrity

The server refuses every bad frame with a specific NACK, discards it up to the next terminator or the 500 ms timeout, and answers the next valid command normally. Table VI maps each kind of bad input to the mechanism and the test that exercises it.

TABLE VI
BAD INPUT, THE SERVER'S HANDLING, AND THE TEST THAT CHECKS IT.

Bad input	Handling and response	Test
Partial frame	dropped after 500 ms of silence; `NACK:TIMEOUT`	`test_partial_frame_times_out_and_is_discarded`; `test_frame_split_across_writes` (a split under 500 ms is reassembled)
Oversized frame	discarded to terminator; one `NACK:TOOLONG`	`test_oversized_frame_rejected_and_tail_not_executed`
Garbage bytes	non-printable byte flags the frame; `NACK:BADCMD`	`test_garbage_bytes_rejected_then_recovers`
Wrong case or unknown command	exact match only; `NACK:BADCMD`	`test_unknown_and_wrong_case_commands`; `test_malformed_move_does_not_touch_state`; `test_malformed_suffix_is_not_a_checksum`
Corrupted bytes	CRC-8 suffix checked before parsing; `NACK:CHECKSUM`, nothing runs (frames that carry a suffix)	`test_wrong_checksum_rejected_and_nothing_executed`; `test_corrupted_payload_is_detected`
UART errors	flags cleared in the interrupt; frame rejected with `NACK:BADCMD`	no dedicated test; first seen through the oversize test

A. A 500 ms timeout is far longer than any frame

A 64-byte frame takes about 5.6 ms on the wire (10 bits per byte at 115200 baud), so a healthy sender never reaches 500 ms between bytes, and a stalled sender is cleared within about half a second. My test checks only that the `NACK:TIMEOUT` arrives inside the 2 s read window of the harness; I did not measure the exact delay.

B. The checksum detects corruption the framer cannot see

Framing only checks the shape of a frame. A flipped bit that turns `MOVE:4` into `MOVE:5` still has the right shape, so the framer accepts it and the server would play the wrong cell. The CRC-8 suffix closes that gap for frames that carry it: `test_corrupted_payload_is_detected` sends exactly that substitution with the old checksum and checks that the board does not change. A CRC with a degree-8 polynomial detects every single-bit error and every burst of up to 8 bits. The protection is optional: a plain frame has none, and damage to the `*` or the two digits makes a checksummed frame look plain, when it is usually (not provably) rejected as an unknown command.

C. The next terminator resynchronises the stream

A bad frame sets `frame_err`, the framer drops the remaining bytes, and the first `\n` or `\r` clears the flag after one NACK. The tests send a bad frame and then a valid command, and check that the valid command gets its normal reply and that the bad frame's tail did not run (Listing 2).

D. No path in the main loop waits without a bound

The framer loop ends when the ring is empty, which is at most 127 bytes. Transmit is polled with a 100 ms timeout per message. The main loop contains no `HAL_Delay`: the button is debounced by comparing timestamps, and the only delays are the three LED flashes (600 ms) at start-up, before any request is accepted. I did not enable the independent watchdog, and `Error_Handler` halts only if the HAL initialisation itself fails.

VII. VALIDATION

A. One fixture puts the board into a known state before every test

The harness uses pytest [7] and pySerial [3]. The port name comes from the `SERIAL_PORT` environment variable, and the suite skips if it is unset, so it cannot fail on a machine without a board. Before each test the fixture sends `CONNECT` until the board answers `OK` (for up to 10 s, so it does not depend on the boot banner or on a reset when the port opens), waits for stale replies, and sends `DISCONNECT`. Every test therefore starts in IDLE with no session. Run the core suite with `python -m pytest -v test_protocol.py`, the extension suite with `test_extras.py`, and the whole suite with a checksum on every request by setting `USE_CRC=1`. The board results in this section come from the real board on `COM14` (Windows, Python 3.11.9, pytest 7.4.3). Results from a PC build of the firmware are labelled as such.

B. 21 of 23 tests passed on the first board run, and the two failures were real

The core suite (`test_protocol.py`) has 23 tests: the required cases of Table VII, the state machine (`NOCONN`, `NOGAME`, `NOTURN`, resign, disconnect) and the robustness cases of Table VI. On the first run on the board, 21 passed and 2 failed in 12.45 s; the extension tests did not exist yet. In `test_out_of_range_rejected` the first four indices (9, 10, 99, -1) returned `NACK:RANGE`, but the fifth, a 15-digit number, returned `NACK:BADCMD`. In the oversize test a 103-character line returned `NACK:BADCMD` instead of `NACK:TOOLONG`. Both failures involve frames much longer than the passing 6-character commands, which led me to the receive path described in Section IV. After I replaced the HAL receive path with the register-level handler, all 47 tests passed on the first full run (27.3 s). They passed again in three further plain runs (27.3 to 27.5 s each) and in one run with a checksum on every request (27.8 s), so all five runs gave 47 of 47.

Table VII shows that the required behaviour was correct on the first run except for very long indices. The out-of-range test failed only on its last value; `NACK:RANGE` for 9, 10, 99 and -1 was already correct.

C. 24 extension tests check the checksum, the AI and undo

Table VIII summarises `test_extras.py`. The AI test plays 20 seeded random games and checks that every reply differs from the previous board by exactly the human's X and one new O, and that the human never wins. The checksum tests include the substitution of `MOVE:4` by `MOVE:5` with the old checksum, and the known CRC-8 value. Setting `USE_CRC=1` reruns all 47 tests with a checksum on every request and verifies every reply.

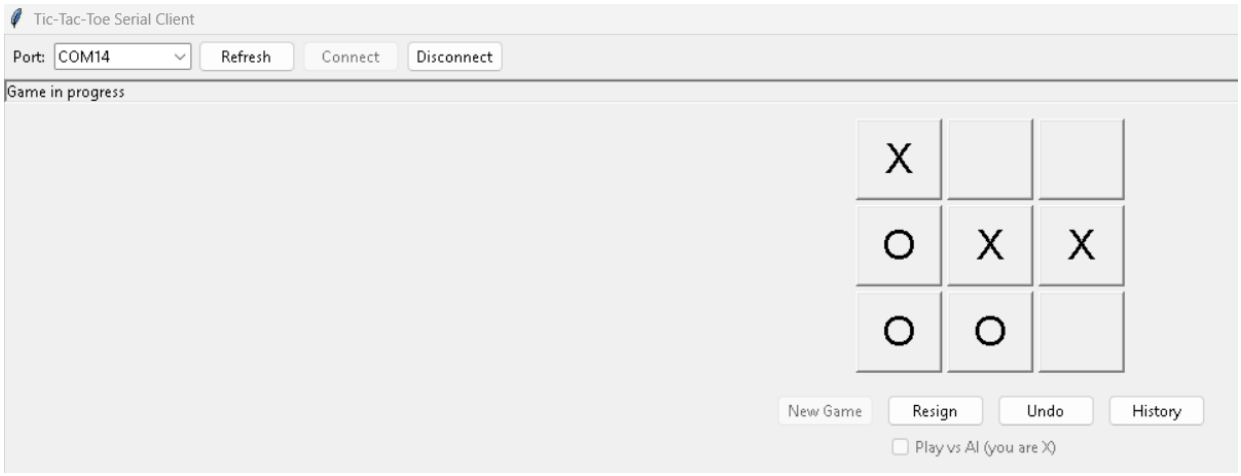


Fig. 5. The GUI connected to the board on COM14 in the middle of a two-player game. X holds cells 0, 4 and 5 and O holds cells 3, 6 and 7, so X is to move. Connect, New Game and the AI tick box are greyed out because the session is open and the server refuses `NEW` during a live game; Resign, Undo and History are active.

TABLE VII
THE BRIEF'S REQUIRED TESTS AND THEIR RESULTS ON THE BOARD.

Requirement (brief, p. 4)	Test name	Run 1	Run 2
`CONNECT` returns `OK`; `NEW` returns an empty board	`test_connect_returns_ok`; `test_new_returns_empty_board`	pass	pass
A legal move changes the board exactly as expected	`test_legal_move_changes_board_exactly`	pass	pass
Occupied cell returns `NACK:OCCUPIED`, board unchanged	`test_occupied_move_rejected_and_board_unchanged`	pass	pass
Out-of-range move returns `NACK:RANGE`	`test_out_of_range_rejected`	fail	pass
Move after game over returns `NACK:NOGAME`	`test_move_after_game_over_rejected`	pass	pass
Scripted win and draw give the correct result	`test_scripted_win_x`; `test_scripted_win_o`; `test_scripted_draw`	pass	pass
AI never makes an illegal move	`test_ai_never_makes_an_illegal_move_and_never_loses` (`test_extras.py`)	n/a	pass

TABLE VIII
EXTENSION TESTS AND THEIR RESULTS ON THE BOARD.

Group	Tests	What they check	Board
Checksum	9	test vector, reply carries a suffix only if the request did, wrong checksum rejected, corrupted payload caught	9/9 pass
AI	7	one AI move per human move, blocks a threat, takes a win, 20 random games with no illegal AI move and no human win	7/7 pass
History, undo	8	move list, undo restores board and turn (both modes), `NOUNDO`, `NOGAME`, `NOCONN`	8/8 pass
All 47, `USE_CRC=1`	47	the whole suite with a checksum on every request	47/47 pass

Figure 5 shows the client in use. The status line reads "Game in progress", and the board shows only the state that the server last confirmed: the client has no way to draw a move that the server did not accept.

D. Code review found five bugs before the board did, and the board found one more

Reading the first firmware against the brief found five defects: the USART1 interrupt was never enabled, so no command was received; `atoi` turned `MOVE:abc` into a move on cell 0; the tail of an oversized line ran as a command; `NEW` was accepted in the middle of a game; and the hand-written boards in the first draw test were wrong, so that test would have failed on correct firmware. I fixed these before the first board run. The board then exposed the receive-path overrun described in Section IV.

VIII. CONCLUSION

The server meets the compulsory requirements: it validates every request in a fixed order before writing state, detects wins and draws, rejects illegal, out-of-state and out-of-turn requests with specific NACKs, and recovers from partial, oversized and garbage input. The evidence is the first board run, where 21 of 23 core tests passed, and after the fix all 47 tests passed on the board in each of five runs. The GUI connects, plays, shows the server's rejection reasons and disconnects. The three optional extensions work behind new commands: a CRC-8 checksum that stops a corrupted but valid-looking command, an AI opponent that cannot make an illegal move, and move history with undo.

Usefulness. The same pattern applies to any small device that must accept commands from an untrusted host, such as a lab instrument or a motor controller: a line-framed protocol with specific error codes, validation before any state change, and a test harness that drives the real device.

Limitations. The checksum is optional, so a client that sends plain frames has no protection, and a bit error in the suffix itself is not provably caught. Undo works only while a game is in progress. One client plays both sides, so

`NACK:NOTURN` is exercised only through the optional mark that the GUI does not send. The receive-path overrun is my diagnosis from test evidence, not a captured flag. I measured no timing on the board: the 500 ms timeout is a design value, and I did not time the AI's search there, although every AI reply arrived inside the harness's 2 s timeout. I did not enable the watchdog. No test covers a win on the ninth move. I ran everything on one board and one Windows PC, and the LED colours on that board are swapped, which one constant corrects in software.

Improvements. A mandatory checksum mode would remove the unprotected plain path. A second client (a second USART and a USB-UART adapter) would make out-of-turn play real, and a hardware timer could add a move clock. The independent watchdog would recover from a firmware fault.

IX. Video

The demonstration video is at https://youtu.be/ySupiEHHqxo, with a copy at https://vt.tiktok.com/ZSg4Cq7YV/. It shows the board and the PC screen together: the GUI connecting, a game with legal and rejected moves, a win or draw, undo and history, a game against the AI, the button and LED, the test suite running, and disconnecting.

X. Code

The source code is at https://github.com/Sibongokuhle-cyber/EEE3095S-TicTacToe-Server. The repository holds the STM32CubeIDE project (`Core/`, `Drivers/`, the `.ioc` file), the Tkinter client (`python_gui/`), the pytest harness (`tests/`) and a README with the directory tree and the build, flash, run and test steps.

References

[1] Department of Electrical Engineering, University of Cape Town, "EEE3095S CS project 2026: Serial game server," Course project brief, 2026.

[2] Python Software Foundation, "tkinter — python interface to Tcl/Tk," https://docs.python.org/3/library/tkinter.html, python 3.11 documentation.

[3] C. Liechti, "pySerial documentation," https://pyserial.readthedocs.io/.

[4] J. Pead, "UCT STM32 dev board schematic, revision v1.06," Course material, University of Cape Town, 2025.

[5] *UM1785: Description of STM32F0 HAL and Low-Layer Drivers*, STMicroelectronics.

[6] *RM0091: STM32F0x1/STM32F0x2/STM32F0x8 Advanced Arm-based 32-bit MCUs, Reference Manual*, STMicroelectronics.

[7] pytest-dev team, "pytest documentation," https://docs.pytest.org/.